



IPB University
— Bogor Indonesia —



Perancangan Keamanan Informasi Tracking Magang (CareerFlow)

Dr. Shelve Nidya Neyman S.Kom., M.Si
Endang Purnama Giri, S.Kom, M.Kom.



Kelompok 5 :

1. Hanifah Syahidah (G6401231067)
2. Khalisa Rana Putri (G6401231141)
3. Aliya Shahira (G6401231116)

Latar Belakang Masalah

Platform pelacakan magang (CareerFlow) menyimpan data sensitif milik pengguna, seperti data pribadi mahasiswa, dokumen CV, kredensial login, hingga riwayat aktivitas. Masalah utama yang harus dicegah adalah akses tidak sah (pembobolan akun), kebocoran data privasi (seperti alamat IP), penyalahgunaan hak akses (mahasiswa memanipulasi lowongan), serta tidak adanya bukti forensik jika terjadi insiden perubahan data secara ilegal.

Risiko tanpa keamanan :

Pencurian Identitas &
Data Breach

Manipulasi Lowongan
Magang

Penyangkalan Aksi
(Repudiation)

Gambaran Sistem

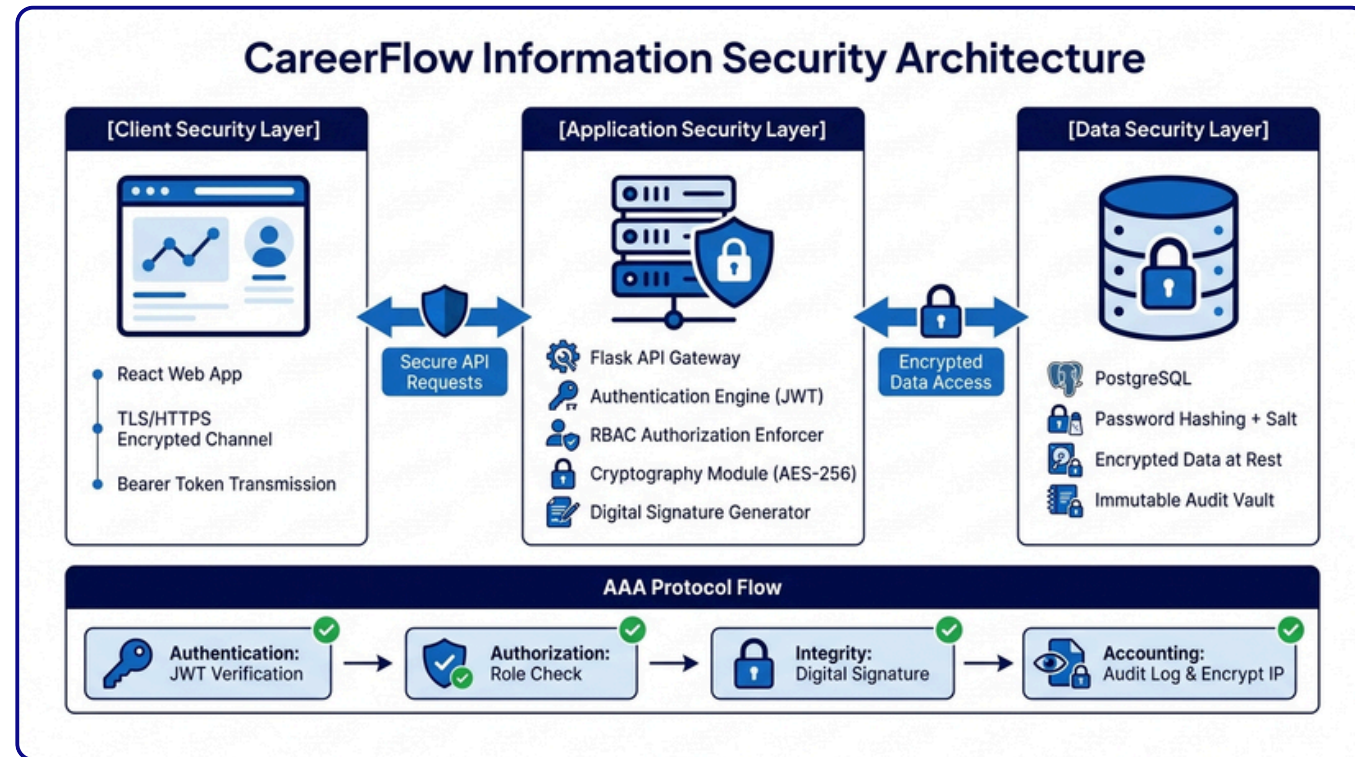
- **Mahasiswa** : Pengguna akhir yang mencari lowongan, mengirim lamaran, dan membuat status lamaran di tracking magang.
- **Admin** : Pengelola sistem yang bertanggung jawab menambah, mengedit, dan menghapus data lowongan magang.



Fitur utama :

- Modul Autentikasi: Signup, Login, Reset Password (via OTP).
- Modul Lowongan (Admin): CRUD (Create, Read, Update, Delete) lowongan.
- Modul Lamaran (Mahasiswa): Filter pencarian, apply magang, pelacakan status.
- Modul Keamanan (Background): Audit Log aktivitas dan enkripsi IP address.

Gambaran Sistem : Arsitektur



Threat Modelling : STRIDE

S

Ancaman: Login paksa sebagai Mahasiswa/Admin

- Mitigasi: Password Hashing (Werkzeug) & Sesi Otentikasi JWT

T

Ancaman: Mengubah informasi lowongan magang secara ilegal.

- Mitigasi: Digital Signature (khusus Admin) & SQLAlchemy ORM.

R

Ancaman: Pelaku menyangkal telah merusak data atau melanggar akses.

- Mitigasi: Immutable Audit Logs (mencatat presisi siapa, apa, dan kapan).

I

Ancaman: Data privasi pelamar (IP Address) terekspos jika database jebol.

- Mitigasi: Enkripsi Ciphertext (AES-256 Fernet) pada Data at Rest.

D

Ancaman: Server down, mahasiswa gagal melamar magang.

- Mitigasi: Isolasi Docker Containers dengan sistem auto-restart.

E

Ancaman: Mahasiswa menembus API untuk menambah/menghapus lowongan.

- Mitigasi: Penegakan Role-Based Access Control (RBAC) via verifikasi JWT.

Kebutuhan Keamanan

Confidentiality

Kerahasiaan data pada sistem database (Data at Rest). Data sensitif harus dienkripsi sebelum dimasukkan ke dalam tabel

Availability

Infrastruktur sistem dirancang menangkal serangan Denial of Service (DoS) sehingga mahasiswa dapat mengakses platform dan mengisi logbook harian tanpa kendala.

Authorization

Sistem Role-Based Access Control (RBAC) untuk membedakan hak antara Admin dan Mahasiswa. Sistem harus menentukan fitur dan data apa saja yang boleh mereka sentuh.

Integrity

Data tetap utuh dan tidak dimanipulasi oleh pihak luar. Implementasi Tanda Tangan Digital (Digital Signature).

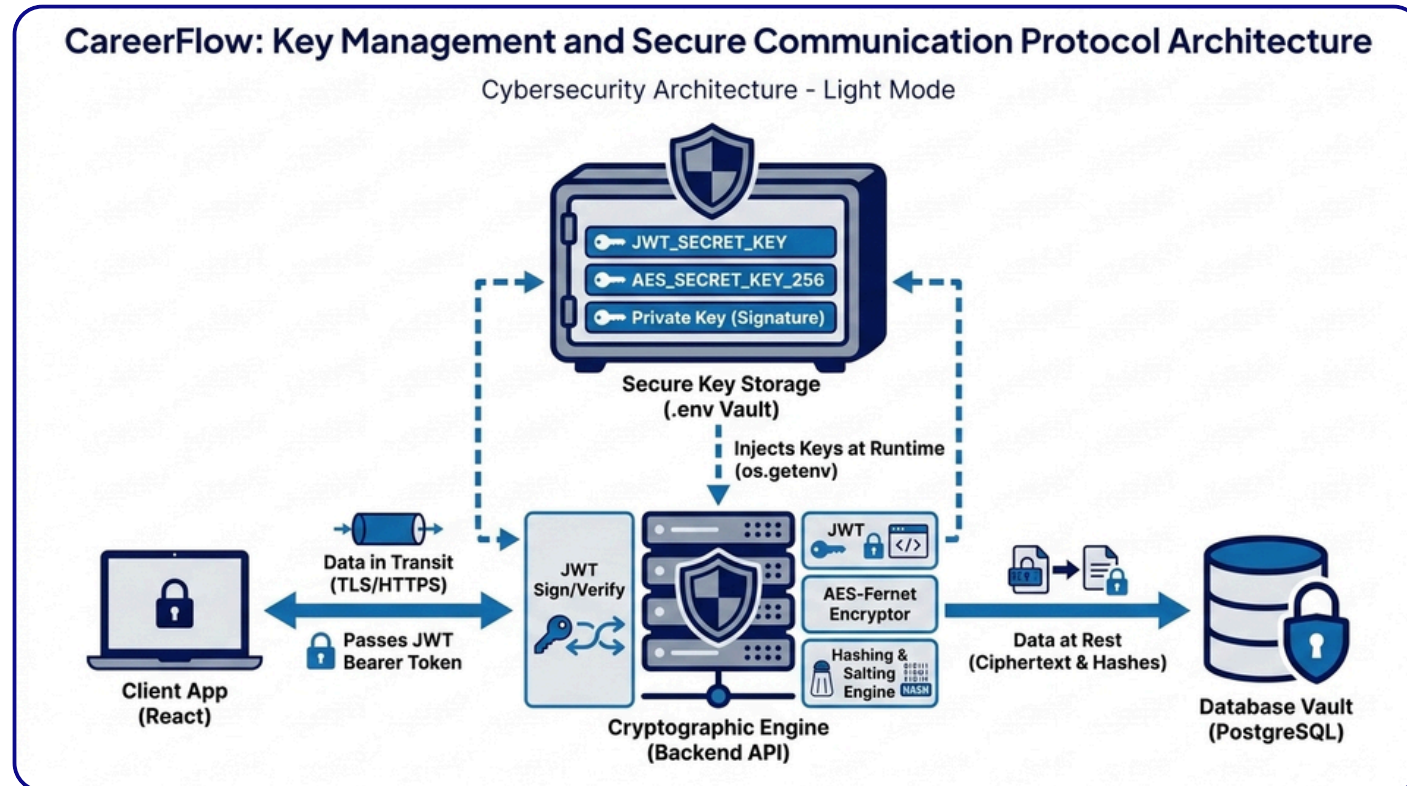
Authentication

Memverifikasi identitas pengguna dan memastikan bahwa pengguna adalah benar-benar orang yang berwenang.

Accountability

Melacak siapa dan melakukan apa dan pada waktu kapan di dalam sistem.

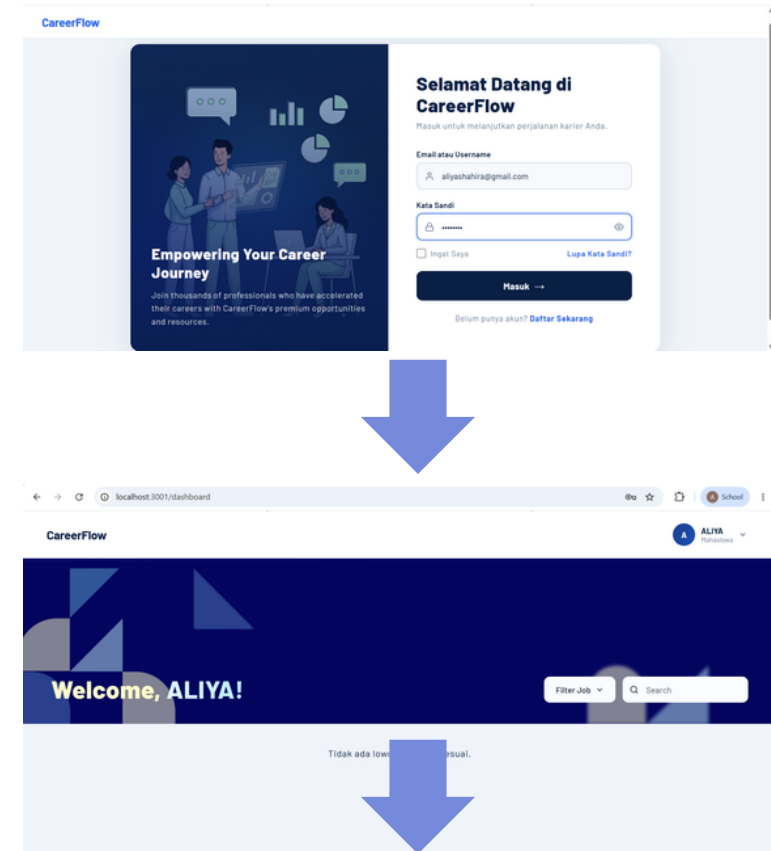
Architecture Diagram: manajemen kunci & komunikasi



DATA AT REST

Fokus & Teknologi Utama: Mengamankan data sensitif (Alamat IP) di tabel database menggunakan algoritma Enkripsi Simetris AES (via cryptography.fernet).

Mekanisme Anti-Retas: Algoritma menggunakan Initialization Vector secara dinamis sehingga hasil sandi (ciphertext) selalu unik dan mencegah peretas melakukan pattern analysis. Kunci utama (Secret Key) diisolasi di file.

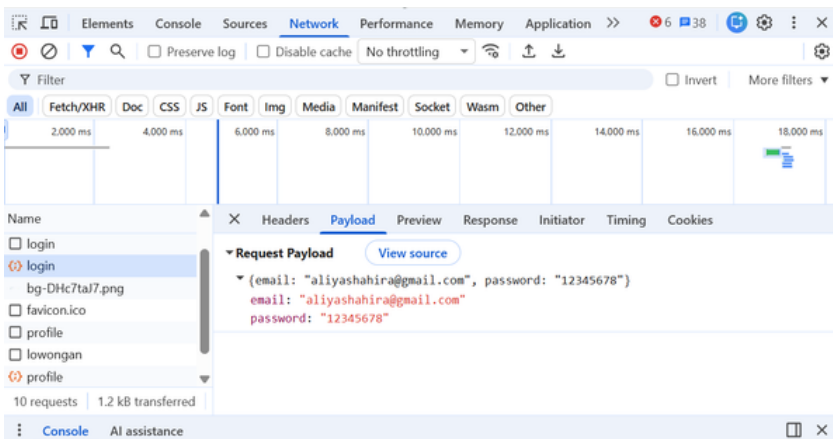


Grid	id	idUser	aksi	status	ipAddress_encrypted	timestamp
1	1	anonymous	[ANONYMOUS] /login - Gagal login menggunakan en	FAILED	gAAAAABqHyqILScR7fwcpDLY2yPm3zng_duGFLBRM	2026-06-02 19:10:00.097
2	2	SHAHIRA	[MAHASISWA] /login - Berhasil melakukan autentikasi	SUCCESS	gAAAAABqHytOp3cFc9Bh7VDOum9rvRt0Wp0buzHE	2026-06-02 19:13:18.082

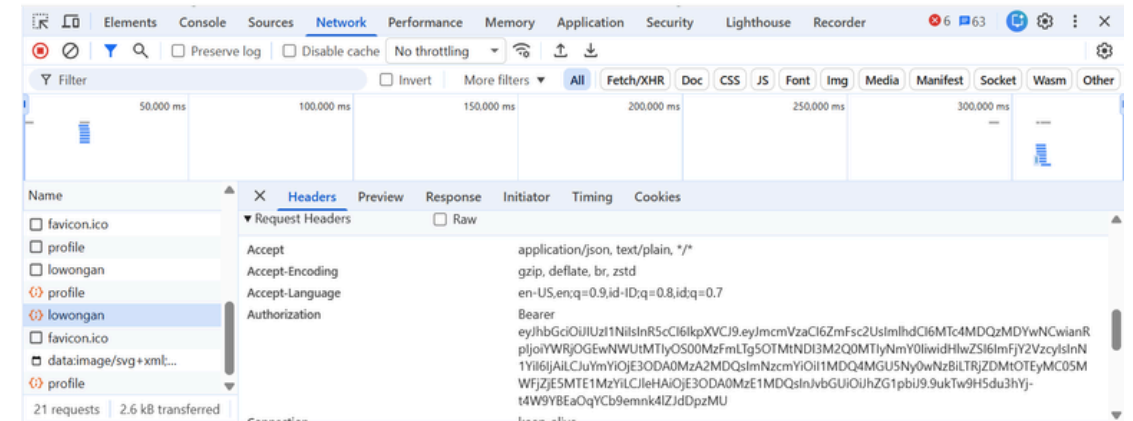
DATA IN TRANSIT

Fokus & Teknologi Utama: Pengamanan data saat transmisi menggunakan HTTPS (TLS) dan JWT Token.

Mekanisme Anti-Retas: Data dienkripsi selama perjalanan dan setiap request divalidasi menggunakan token autentikasi untuk mencegah penyadapan dan akses ilegal.



- Data dikirim dari client → server
- Data masih terlihat dalam bentuk plain text (saat development)
- Di production diamankan dengan HTTPS (TLS)
- Data akan terenkripsi saat dikirim



Authorization: Bearer <JWT Token>

- Token dikirim setiap request yang butuh login
- Berfungsi sebagai identitas user
- Digunakan untuk verifikasi ke server

Manfaat :

- Mencegah akses dari user tidak sah
- Token hanya dimiliki user yang sudah login
- Dikirim melalui HTTPS agar tidak bisa disadap

Authentication

 A-Z nim	A-Z programStudi	A-Z idUser	A-Z nama		
fah	sdsd	37ffd9fd-6046-4b29-a3a8-6884618bfa89	hani		
A-Z email	A-Z password				
fahh4317@gmail.com	scrypt:32768:8:1\$Vp2feehrh9yJ5sxN\$2da64c198f9f39439a7ebff79e				
  id	A-Z email	A-Z otp	A-Z purpose	 created_at	<input checked="" data-bbox="1735 696 1755 705" type="checkbox"/> is_used
1	fahh4317@gmail.com	610053	signup	2026-06-02 16:27:57.417	[v]
34	fahh4317@gmail.com	986864	forgot	2026-06-02 18:35:40.095	[v]

- **Proses Login:** Pengecekan kredensial di endpoint /login secara berurutan (Admin → Mahasiswa).
- **Password Hashing + Salt:** Menggunakan generate_password_hash (Werkzeug) untuk mencegah serangan Rainbow Table. Verifikasi menggunakan check_password_hash.
- **MFA (OTP Email):** Pengamanan ekstra via kode OTP (kadaluarsa 10 menit) untuk proses Signup dan Reset Password.

Authorization

```
< ▶ Promise {<pending>}  
✖ ▶ POST http://localhost:5001/admin/lowongan 403 (FORBIDDEN)  
  Status Code: 403 ▶ {message: 'Akses ditolak, khusus admin'}  
>
```

- **Role-Based Access Control (RBAC):** Hak akses pengguna (Admin/Mahasiswa) disematkan langsung ke dalam Payload Token JWT.
- **Penegakan Akses Endpoint:** Pengecekan role di setiap fungsi (misal: `if role !== 'admin'`). Akses ilegal otomatis ditolak (HTTP 403 Forbidden) dan langsung direkam sebagai insiden.

AES

activity_logs | Enter a SQL expression to filter results (use Ctrl+Space)

	id	idUser	aksi	status	ipAddress_encrypted	timestamp
1	1	SHAHIRA	[MAHASISWA] /login - Berhasil melakukan autentikasi	SUCCESS	gAAAAABqHz7jR6YOxblRH83jVQqjk2p7BQC4_R3b1T	2026-06-02 20:36:51.108
2	2	0	[ADMIN] /login - Berhasil melakukan autentikasi login	SUCCESS	gAAAAABqHz88fUCQ457Txqdx08L2TeZuEUyv6HTON	2026-06-02 20:38:20.297
3	3	SHAHIRA	[MAHASISWA] /login - Berhasil melakukan autentikasi	SUCCESS	gAAAAABqH0VWmi_oSmHHRUyXDEW27pW3I_Uc_6C	2026-06-02 21:04:22.870

Saat alamat IP masuk ke mesin AES, data diacak berulang-ulang sebanyak 10 hingga 14 putaran (tergantung panjang kunci).

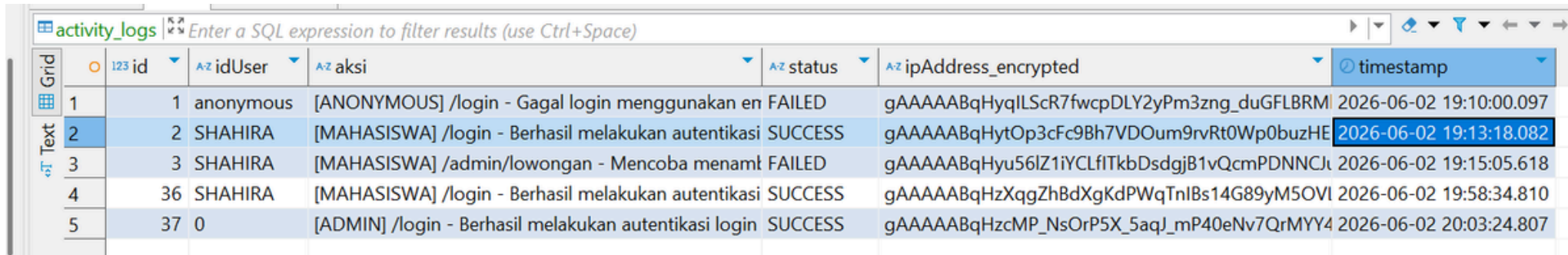
Di dalam setiap putaran, data akan mengalami 4 tahap pengacakan :

1. **SubBytes (Substitusi):** Menukar setiap karakter asli dengan karakter lain secara acak berdasarkan tabel rujukan rahasia (S-box).
2. **ShiftRows (Geser Baris):** Menggeser posisi letak karakter.
3. **MixColumns:** Mencampur karakter dalam kolom yang sama menggunakan operasi matematika.
4. **AddRoundKey (Gembok Kunci):** Mengunci hasil acakan di atas menggunakan sebagian dari AES_SECRET_KEY.

Tahap Modifikasi AES

- Memodifikasi kelas ActivityLog. Merapikan nama kolom (memastikan penulisan idUser, aksi, ipAddress_encrypted, dan timestamp).
- auth_service.py & auth_controller.py (Logika Backend)
- Menyesuaikan fungsi pencatatan log (log_activity) agar cocok dengan tabel models.py yang baru. Data role dan endpoint kini digabungkan ke dalam satu kalimat di kolom aksi (contoh: [MAHASISWA] /login - Berhasil...).
- Menambahkan baris .env di bagian paling bawah agar kunci rahasia (seperti AES_SECRET_KEY dan DB_PASSWORD) tidak ikut terbang ke GitHub.

Accounting



The screenshot shows a database interface with a table named 'activity_logs'. The table has columns: id, idUser, aksi, status, ipAddress_encrypted, and timestamp. The data is as follows:

	id	idUser	aksi	status	ipAddress_encrypted	timestamp
1	1	anonymous	[ANONYMOUS] /login - Gagal login menggunakan en	FAILED	gAAAAABqHyqILScR7fwcpDLY2yPm3zng_duGFLBRM	2026-06-02 19:10:00.097
2	2	SHAHIRA	[MAHASISWA] /login - Berhasil melakukan autentikasi	SUCCESS	gAAAAABqHytOp3cFc9Bh7VDOum9rvRt0Wp0buzHE	2026-06-02 19:13:18.082
3	3	SHAHIRA	[MAHASISWA] /admin/lowongan - Mencoba menamk	FAILED	gAAAAABqHyy56lZ1iYCLfITkbDsdgjb1vQcmPDNNCJk	2026-06-02 19:15:05.618
4	36	SHAHIRA	[MAHASISWA] /login - Berhasil melakukan autentikasi	SUCCESS	gAAAAABqHzXqgZhBdXgKdPWqTnlBs14G89yM5OVI	2026-06-02 19:58:34.810
5	37	0	[ADMIN] /login - Berhasil melakukan autentikasi login	SUCCESS	gAAAAABqHzcMP_NsOrP5X_5aqJ_mP40eNv7QrMY4	2026-06-02 20:03:24.807

Fungsi Utama: Bertindak sebagai Audit Log untuk melacak aktivitas sistem, memastikan Non-Repudiation dan mempermudah investigasi keamanan.

Skenario Login History:

- Sukses: Mencatat identitas sah (contoh: "[MAHASISWA] /login").
- Gagal: Mencatat upaya login tak dikenal sebagai "anonymous" dengan status "FAILED" untuk mendeteksi ancaman Brute Force.

DIGITAL SIGNATURE

Digital signature dibuat melalui kelas DigitalSignatureService pada services/signature_service.py. Mekanisme digunakan saat:

- add_lowongan (menambahkan lowongan baru)
- update_lowongan (memperbarui lowongan)

Perbaikan fungsi to_dict() pada model database bertindak sebagai verifikator menggunakan Public Key untuk memastikan status data ("AMAN / VALID" atau "PERINGATAN: DATA TELAH DIMANIPULASI!")

Data lowongan yang krusial dikonversi menjadi format JSON yang konsisten (_canonical_json) lalu ditandatangani menggunakan private key sehingga menghasilkan nilai signature.

```
# Kolom untuk menyimpan digital signature (hex string panjang)
signature = db.Column(db.Text, nullable=True)
```

? Kenapa menggunakan digital signature:

- Integritas data: memastikan lowongan magang tidak dimanipulasi oleh pihak ketiga sejak lowongan dibuat oleh Admin
- Non-repudiation: membuktikan secara sah bahwa lowongan benar-benar diterbitkan oleh Admin yang memegang kunci privat tersebut dan tidak bisa disangkal

RSA-2048

```
self.private_key = rsa.generate_private_key(  
    public_exponent=65537,  
    key_size=2048  
)  
self.public_key = self.private_key.public_key()
```

RSA dipakai sebagai algoritma asymmetric key (private key untuk menandatangani dan public key untuk memverifikasi). Key size 2048 bit dipilih karena standar minimum industri yang sudah dianggap aman dari serangan brute-force modern. Public exponent 65537 adalah angka prima Fermat yang umum digunakan sebagai standar industri. Angka ini dipilih karena mempercepat proses kalkulasi matematis dalam verifikasi kunci publik tanpa mengurangi aspek keamanan dari RSA.

*dalam dunia nyata, private key harus digenerate sekali dan disimpan aman (bukan digenerate ulang tiap restart), tapi untuk memudahkkan development kita generate di memori saja

SHA-256

```
padding.PSS(  
    mgf=padding.MGF1(hashes.SHA256()),  
    salt_length=padding.PSS.MAX_LENGTH  
)  
hashes.SHA256()
```

- Digunakan di dalam argumen padding PSS dan MGF1 (hashes.SHA256()). sebelum data lowongan ditandatangani RSA, diringkas dulu pake nilai hash berukuran tetap (256-bit)
- Algoritma RSA sangat lambat jika harus menandatangani dokumen berukuran besar secara langsung. Oleh karena itu, kita ringkas pake SHA-256 sebelum ditandatangani RSA
- SHA-256 juga bersifat collision resistance
- Padding PSS adalah skema padding berbasis probalistik (menambahkan salt sebelum dihash) karena lebih aman secara teoritis dan matematis

Hasil Pengujian



Functional Testing

Skenario : Login menggunakan akun valid & fitur Tambah Lowongan.

Hasil : Sistem berhasil memverifikasi kredensial dan add lowongan ke dalam database



Security Testing

- Otentikasi: Input password salah ditolak dengan peringatan HTTP 401 Unauthorized.
- Otorisasi (RBAC): Simulasi mahasiswa memaksa akses API Add Lowongan (Admin) menggunakan Console Browser → Hasil: Sistem memblokir dengan HTTP 403 Forbidden.
- Kriptografi: Inspeksi database (via DBeaver) → Hasil: Kolom password terbukti di-hash, dan kolom IP Address terbukti menjadi ciphertext (AES-256).



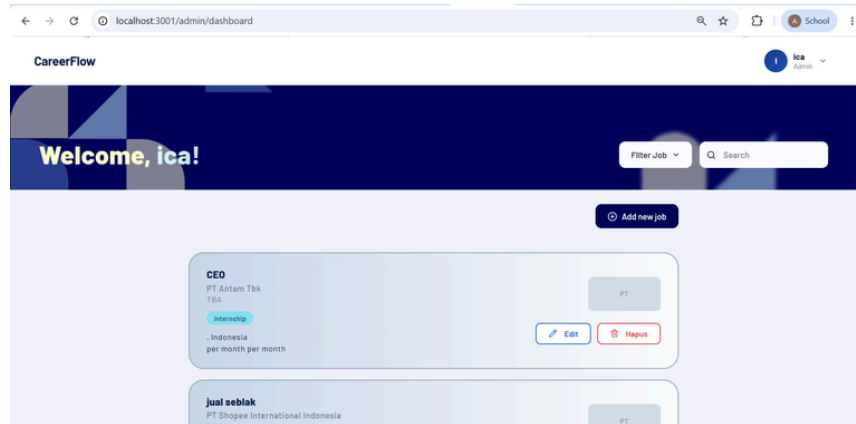
Integration Testing

Skenario: Pengiriman Token JWT dan pencatatan Audit Log.

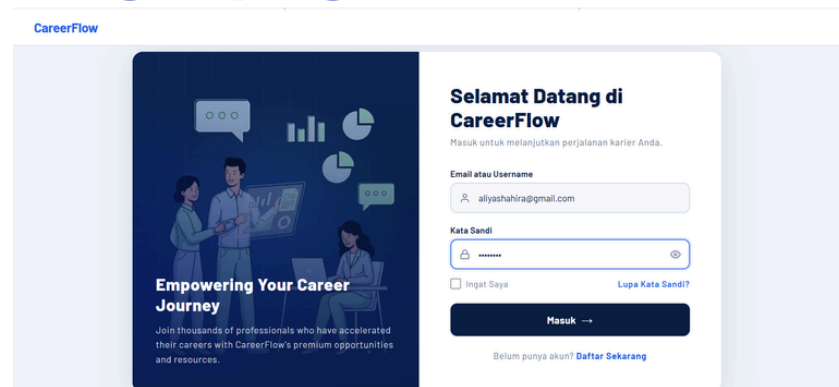
Hasil : (Frontend) Token JWT di HTTP Headers, Flask (Backend), validasi akses, INSERT Postgre



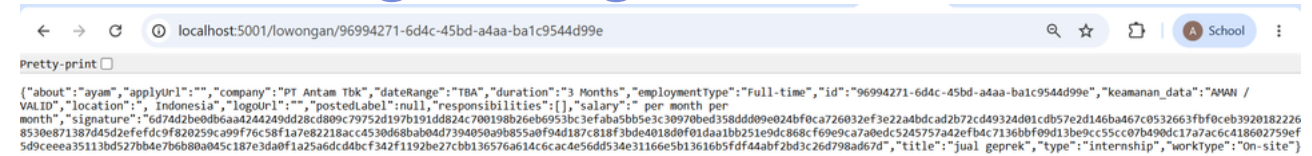
Akun Admin



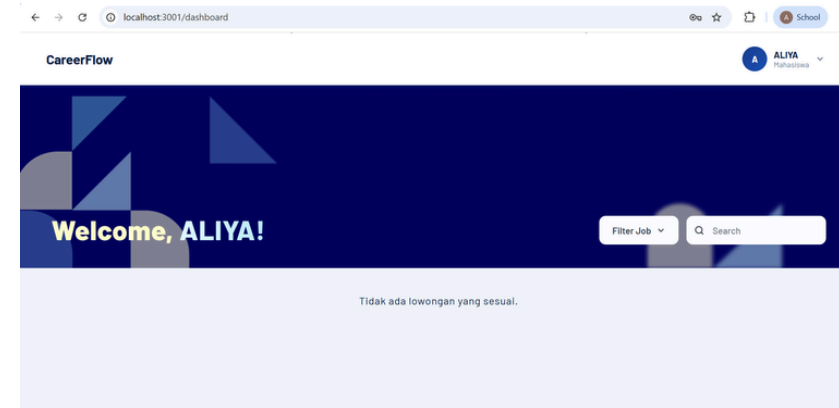
login page



Validasi Digital Signature



Akun mahasiswa



Validasi Digital Signature

Hasil Signature

A-Z signature ▼
160bf0c69ee46912c1201f3ad938f910e51

Sebelum Tampering (Masih Aman)

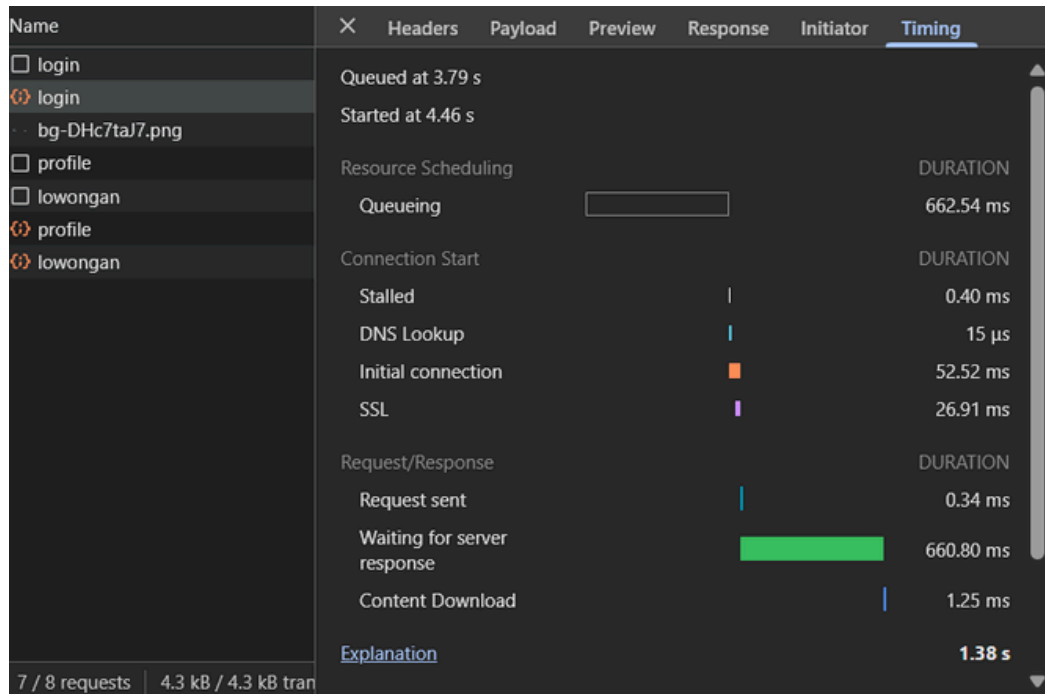
```
{"about": "Aduh udahan gasi proyeknya", "applyUr  
384966f6d86f", "keamanan_data": "AMAN / VALID",  
...}
```

Sesudah Tampering

```
{"about": "NOW AKU SUKA KI", "applyUr1": "", "company": "PT Shopee Internat  
384966f6d86f", "keamanan_data": "PERINGATAN: DATA TELAH DIMANIPULASI!", "  
month", "signature": "160bf0c69ee46912c1201f3ad938f910e516e10a1f01a69880
```


SEBELUM IMPLEMENTASI KEAMANAN

Response Time dan Latency :

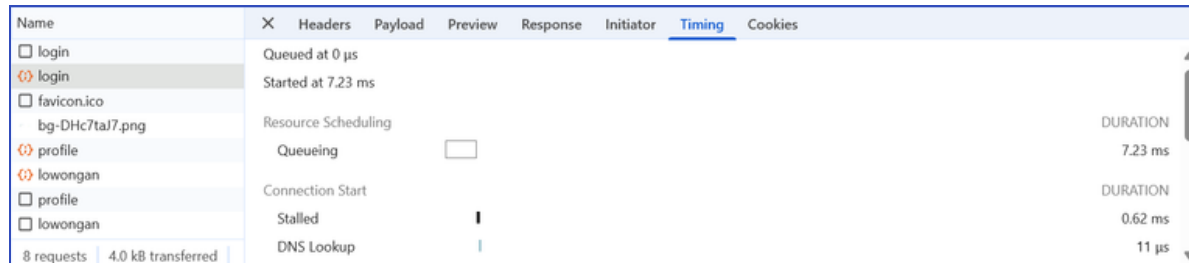
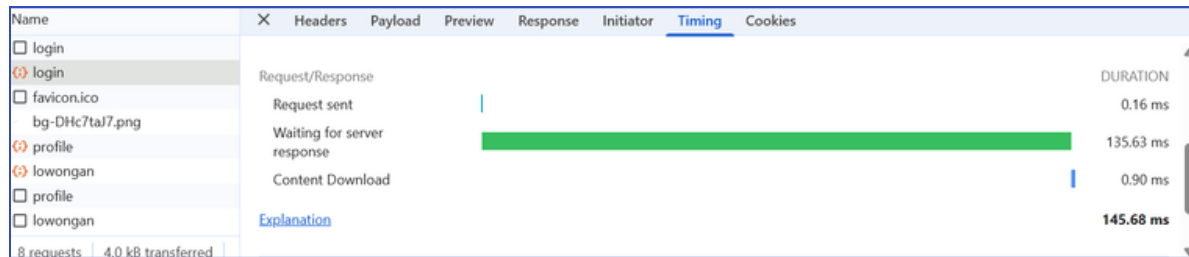


CPU dan Ukuran Data:

CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O	PIDS
d526a1481bf1	careerflow-frontend	0.00%	13.22MiB / 7.619GiB	0.17%	10.2kB / 1.84MB	3.27MB / 61.4kB	17
3dcd6c14bae2	careerflow-backend	0.02%	56.99MiB / 7.619GiB	0.73%	54.8kB / 50.1kB	25.6MB / 168kB	3
1552298739af	careerflow-db	0.00%	31.13MiB / 7.619GiB	0.40%	69.3kB / 359kB	17.2MB / 889kB	9

SESUDAH IMPLEMENTASI KEAMANAN

Response Time dan Latency :



CPU dan Ukuran Data:

CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O	PIDS
9dfa54c12776	careerflow-frontend	0.00%	12.66MiB / 7.60GiB	0.16%	1.04kB / 126B	1.06MB / 8.19kB	17
3355dda434e5	careerflow-backend	0.01%	55.65MiB / 7.60GiB	0.71%	4.46kB / 4.78kB	5MB / 238kB	3
3ccb7093ae24	careerflow-db	0.03%	20.5MiB / 7.60GiB	0.26%	6.08kB / 3.31kB	8.97MB / 225kB	7



KESIMPULAN

PENCAPAIAN TUJUAN

- penerapan password hashing untuk mencegah serangan rainbow table, autentikasi, dan otorisasi berbasis token JWT
- enkripsi data sensitif pada data at rest menggunakan algoritma AES-256 Fernet
- penerapan sigital signature berbasis RSA-2048 dan SHA-256

KETERBATASAN SISTEM

Pembuatan sepasang kunci enkripsi (private dan public key) masih dilakukan secara dinamis di dalam memori setiap kali sistem melakukan restart.

Keterbatasan ini perlu diperbaiki karena kunci privat harusnya digenerate sekali dan disimpan secara permanen di tempat penyimpanan kunci yang aman

PENGEMBANGAN BERIKUTNYA

Sistem perlu meningkatkan manajemen kunci agar lebih aman dan tidak bergantung pada penyimpanan sementara saat runtime.

Selain itu, seluruh komunikasi antara klien dan server harus menggunakan protokol HTTPS (TLS) untuk melindungi data.

Ketersediaan layanan juga dapat ditingkatkan melalui optimasi konfigurasi Docker, sperti mekanisme auto-restart.



IPB University
— Bogor Indonesia —

Inspiring Innovation with Integrity
in Agriculture, Ocean and Biosciences for a Sustainable World